FIAP

Faculdade de Informática e Administração Paulista

PROJETO CLYVOVET

SPRINT 4

Mastering Relational and Non-Relational Database

INTEGRANTES

Gustavo Barrios de Araújo | RM563358

Matheus Almeida Ribeiro | RM562980

Phietro Solon Oliveira | RM563842

Turma: 2TDSPW

Repositório GitHub: https://github.com/PhietroSolonoo/ClyvoVet-Sprint4DB.git

São Paulo – SP

2026

1. Introdução

O ClyvoVet é uma plataforma desenvolvida com o objetivo de centralizar, organizar e facilitar o gerenciamento do histórico de saúde de animais de estimação. A solução permite armazenar informações referentes aos tutores, pets, clínicas veterinárias, profissionais responsáveis, consultas, vacinas, medicamentos, exames e alertas, proporcionando maior rastreabilidade e organização dos registros.

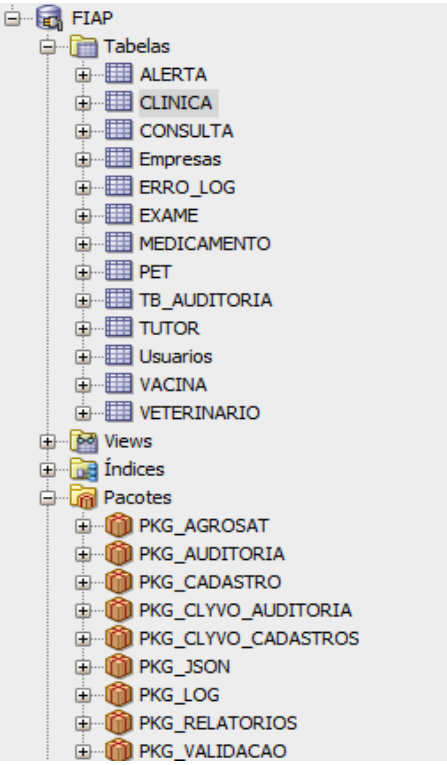
Durante a Sprint 4, o projeto foi aprimorado com foco na integração entre bancos de dados relacionais e não relacionais, utilizando Oracle Database e MongoDB, além do desenvolvimento de uma API em C# com .NET para estabelecer a comunicação entre a aplicação e o banco relacional.

2. Modelagem do Banco de Dados Relacional

A estrutura relacional foi implementada no Oracle Database, sendo composta por 11 tabelas: TUTOR, PET, CLINICA, VETERINARIO, CONSULTA, VACINA, MEDICAMENTO, EXAME, ALERTA, ERRO_LOG e TB_AUDITORIA.

A modelagem foi organizada para garantir a integridade referencial, reduzir a redundância de informações e estabelecer relacionamentos consistentes entre as entidades. Foram utilizadas chaves primárias e estrangeiras para assegurar a identificação dos registros e a associação correta entre os dados armazenados.

Adicionalmente, foi implementada uma Trigger de Auditoria responsável por registrar operações de INSERT, UPDATE e DELETE na tabela TB_AUDITORIA. Esse mecanismo permite acompanhar alterações realizadas nos dados, contribuindo para a rastreabilidade, a identificação de operações e o controle das modificações efetuadas no sistema.



3. Organização e Modularização com Packages PL/SQL

Para melhorar a organização, manutenção e reutilização do código, as rotinas PL/SQL foram refatoradas e distribuídas em seis Packages, cada um responsável por funcionalidades específicas do sistema.

O PKG_VALIDACAO centraliza as regras de validação dos dados, enquanto o PKG_LOG organiza os mecanismos de registro de erros. O PKG_CADASTRO reúne as operações de cadastro, e o PKG_JSON é responsável pela transformação de informações relacionais em documentos JSON.

Complementarmente, o PKG_RELATORIOS concentra as rotinas de geração de relatórios e cálculos agregados, enquanto o PKG_AUDITORIA organiza as funcionalidades relacionadas ao acompanhamento das operações realizadas no banco.

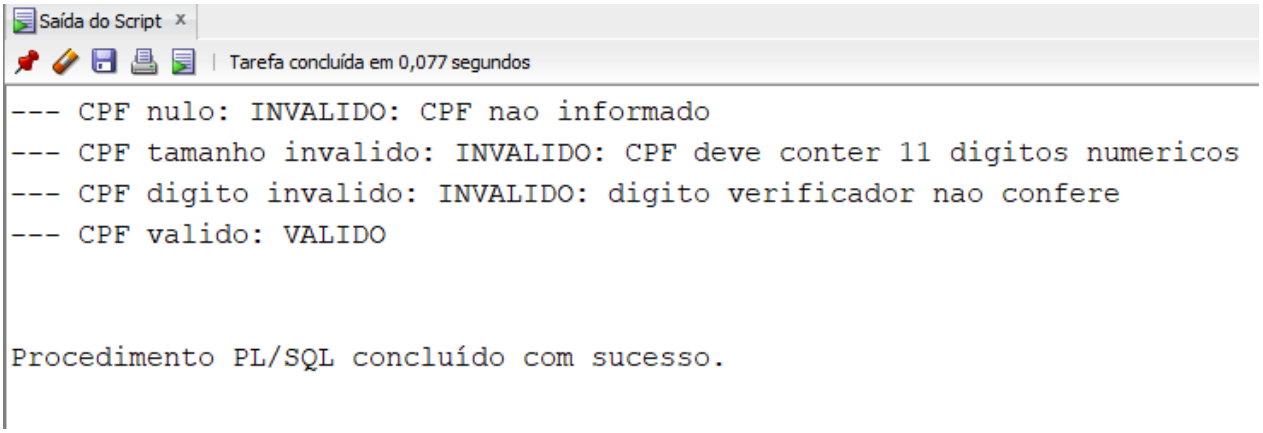
Essa divisão proporciona maior modularidade, facilita a identificação de problemas e reduz a duplicação de código, permitindo que cada componente seja desenvolvido, testado e mantido de maneira independente.

4. Tratamento de Exceções

As rotinas foram estruturadas com tratamento de exceções para lidar com entradas inválidas, inconsistências de dados e possíveis falhas durante a execução dos procedimentos.

Foram consideradas pelo menos três condições de exceção por rotina, conforme os requisitos estabelecidos, incluindo situações como e_cpf_nulo, utilizada para identificar CPFs não informados; e_tamanho_invalido, responsável por sinalizar valores com quantidade incorreta de caracteres; e e_cpf_repetido, aplicada para impedir cadastros duplicados.

O tratamento dessas situações permite retornar mensagens específicas e compreensíveis, evitando a continuidade de operações inválidas e contribuindo para a consistência das informações armazenadas.



5. Conversão Manual de Dados Relacionais para JSON

A conversão dos dados relacionais para o formato JSON foi implementada por meio do Package PKG_JSON, sem a utilização de funções nativas de geração de JSON do Oracle, como JSON_OBJECT, JSON_ARRAYAGG ou funcionalidades equivalentes.

A construção dos documentos foi realizada manualmente utilizando recursos de PL/SQL, concatenação de strings e organização explícita dos atributos e valores retornados pelas consultas ao banco.

Essa abordagem permite compreender detalhadamente o processo de transformação de registros relacionais em estruturas documentais, incluindo a representação de atributos, objetos e coleções de informações relacionadas. A funcionalidade também possibilita disponibilizar os dados dos pets em um formato adequado para consumo por aplicações externas e integração com bancos de dados não relacionais.

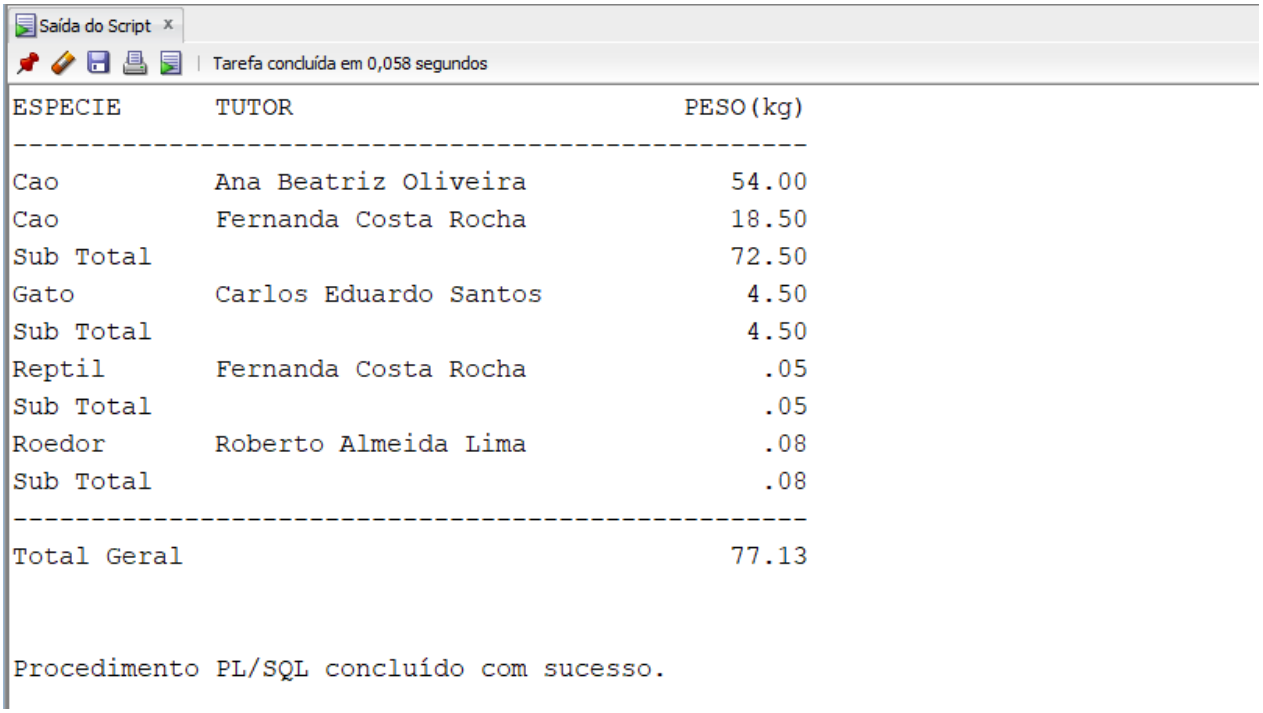
6. Relatórios com Agregações Manuais

O Package PKG_RELATORIOS foi desenvolvido para realizar operações de agrupamento e consolidação de informações sem utilizar recursos automáticos de agregação hierárquica, como ROLLUP e CUBE.

Entre as funcionalidades implementadas, destaca-se a procedure PRC_RELATORIO_PESO_POR_ESPECIE, responsável por calcular o somatório dos pesos dos animais agrupados por espécie e tutor.

O processamento foi estruturado para controlar os acumuladores, identificar as mudanças de agrupamento e apresentar os subtotais correspondentes, além do total geral calculado ao final da execução.

Nos testes realizados, o relatório apresentou um Total Geral de 77,13, demonstrando o resultado do processamento dos registros utilizados na base de testes. Essa implementação evidencia a aplicação de lógica procedural no processamento de dados.


Saída do Script
Tarefa concluída em 0,058 segundos
ESPECIE TUTOR PESO(kg)
Cao Ana Beatriz Oliveira 54.00
Cao Fernanda Costa Rocha 18.50
Sub Total 72.50
Gato Carlos Eduardo Santos 4.50
Sub Total 4.50
Reptil Fernanda Costa Rocha .05
Sub Total .05
Roedor Roberto Almeida Lima .08
Sub Total .08
Total Geral 77.13
Procedimento PL/SQL concluído com sucesso.

7. Desenvolvimento da API e Integração com Oracle

Para disponibilizar as funcionalidades do banco relacional a aplicações externas, foi desenvolvida uma API REST utilizando C# e a plataforma .NET Web API.

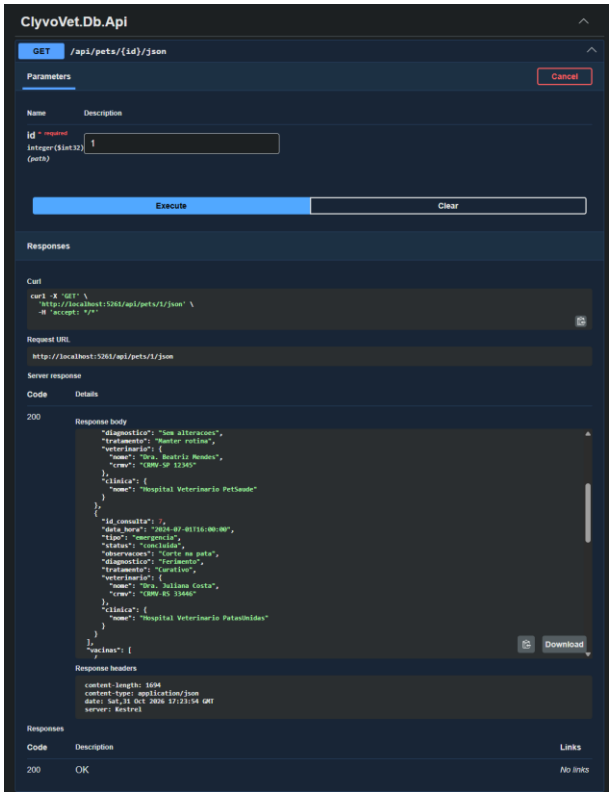
A comunicação com o Oracle Database foi estabelecida por meio da biblioteca Oracle.ManagedDataAccess.Core, utilizando ADO.NET para executar as procedures disponibilizadas pelos Packages PL/SQL.

A implementação foi realizada sem a utilização do Entity Framework, permitindo o controle direto das conexões, dos comandos executados e dos parâmetros enviados ao banco de dados.

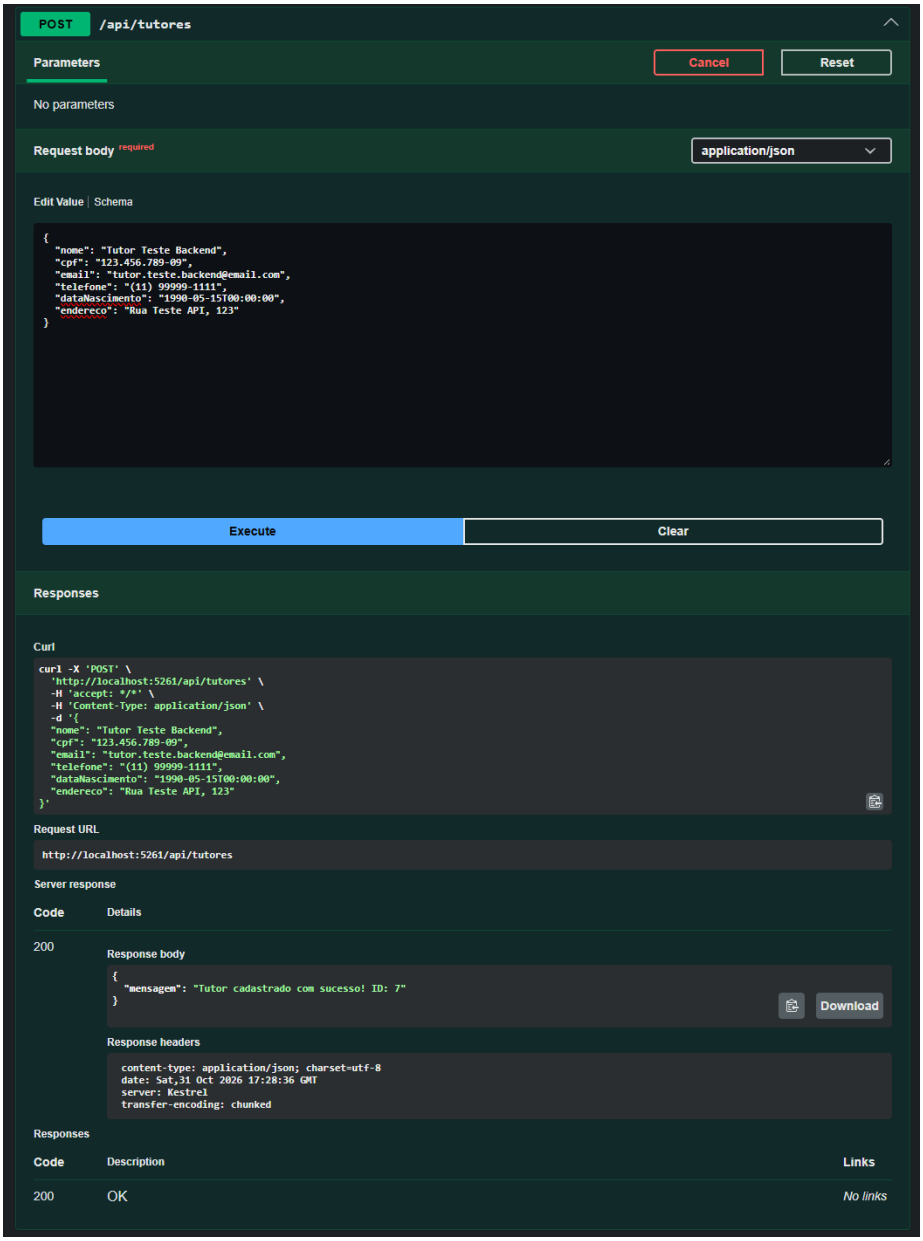
A API disponibiliza operações de cadastro de tutores e consulta de informações de pets no formato JSON. Entre os endpoints implementados, destacam-se POST /api/tutores, responsável pelo cadastro de novos tutores, e GET /api/pets/{id}/json, utilizado para recuperar os dados estruturados de determinado animal.

8. Testes e Segurança da Integração

A validação das operações foi realizada por meio da interface Swagger, permitindo executar requisições HTTP e verificar as respostas retornadas pela API. No teste de consulta, o endpoint GET /api/pets/1/json retornou o documento JSON correspondente ao pet Rex, demonstrando a integração entre a aplicação C# e a rotina de geração de JSON implementada no Oracle.



Também foi validada a operação de cadastro por meio do endpoint POST /api/tutores, cujo retorno apresentou a mensagem "Tutor cadastrado com sucesso! ID: 7", confirmando a execução da rotina de cadastro no banco de dados.



Em relação à segurança, as credenciais de acesso ao Oracle foram mantidas fora do código-fonte versionado, utilizando variáveis de ambiente e configurações protegidas por meio do arquivo .gitignore. Essa prática reduz o risco de exposição de informações sensíveis no repositório GitHub.

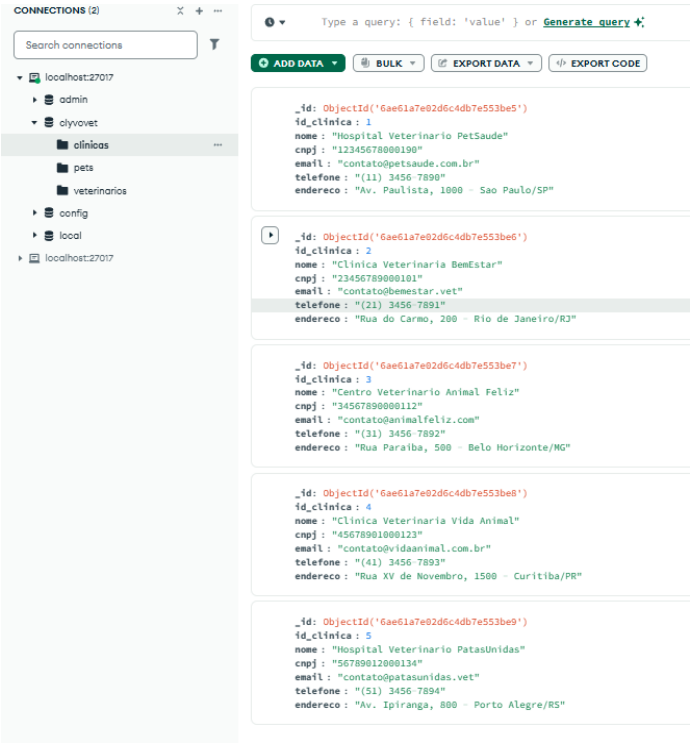
9. Implementação do Banco de Dados NoSQL

Como parte da integração entre diferentes paradigmas de armazenamento, os dados exportados do Oracle em formato JSON foram importados para o MongoDB, um sistema gerenciador de banco de dados não relacional orientado a documentos.

A modelagem adotou uma arquitetura híbrida, combinando documentos embutidos (embedding) e referências entre coleções (referencing). Os documentos da coleção pets incorporam informações pertinentes ao histórico de saúde dos animais, permitindo consultar dados relacionados de forma agrupada.

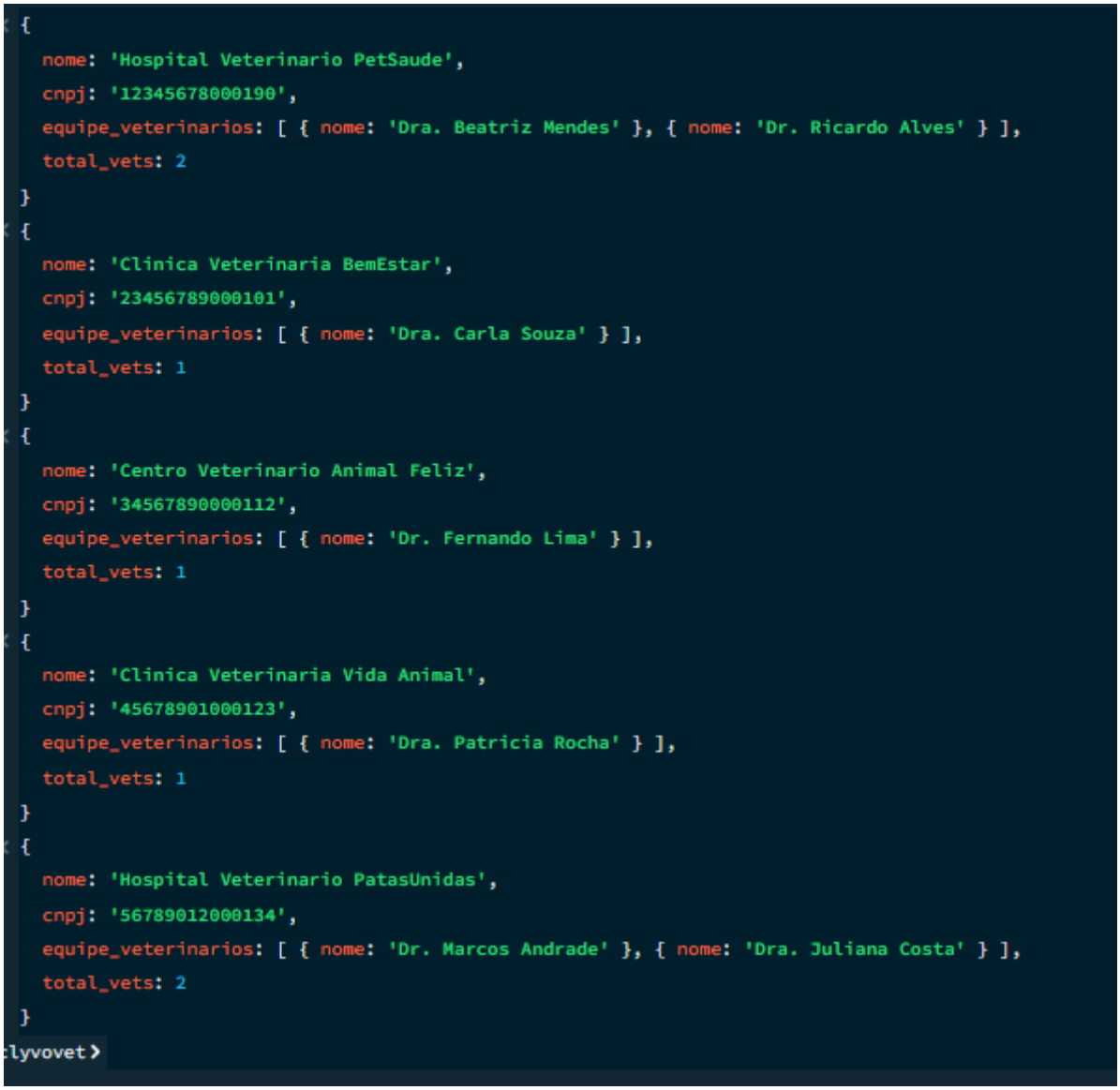
As entidades clínicas e veterinários foram organizadas em coleções próprias, clinicas e veterinarios, com referências entre os documentos. Essa estratégia evita replicação desnecessária de informações compartilhadas e favorece a manutenção de dados cadastrais independentes.

A escolha do modelo híbrido considera os padrões de acesso: informações consultadas em conjunto com o pet podem ser incorporadas ao documento, enquanto entidades compartilhadas por diferentes registros permanecem referenciadas.



10. Consultas e Relacionamentos no MongoDB

Para demonstrar a recuperação de informações relacionadas entre coleções, foi utilizada uma consulta de agregação com o estágio $lookup, associando documentos de clínicas aos respectivos veterinários. A operação evidencia a possibilidade de realizar junções lógicas em um banco orientado a documentos quando a modelagem emprega referências.


{
nome: 'Hospital Veterinario PetSaude',
cnpj: '12345678000190',
equipe_veterinarios: [{ nome: 'Dra. Beatriz Mendes' }, { nome: 'Dr. Ricardo Alves' }],
total_vets: 2
}
{
nome: 'Clinica Veterinaria BemEstar',
cnpj: '23456789000101',
equipe_veterinarios: [{ nome: 'Dra. Carla Souza' }],
total_vets: 1
}
{
nome: 'Centro Veterinario Animal Feliz',
cnpj: '34567890000112',
equipe_veterinarios: [{ nome: 'Dr. Fernando Lima' }],
total_vets: 1
}
{
nome: 'Clinica Veterinaria Vida Animal',
cnpj: '45678901000123',
equipe_veterinarios: [{ nome: 'Dra. Patricia Rocha' }],
total_vets: 1
}
{
nome: 'Hospital Veterinario PatasUnidas',
cnpj: '56789012000134',
equipe_veterinarios: [{ nome: 'Dr. Marcos Andrade' }, { nome: 'Dra. Juliana Costa' }],
total_vets: 2
}
lyvovet>

11. Conclusão

A Sprint 4 do projeto ClyvoVet consolidou a integração entre diferentes tecnologias de persistência e acesso a dados, reunindo Oracle Database, PL/SQL, C#/.NET Web API e MongoDB em uma solução voltada ao gerenciamento do histórico de saúde de animais de estimação. A organização das rotinas em Packages, a implementação de validações e mecanismos de auditoria, a construção manual de documentos JSON e de relatórios agregados, bem como a comunicação da API com o banco relacional, demonstram a aplicação prática dos conceitos estudados na disciplina Mastering Relational and Non-Relational Database. A importação dos documentos para o MongoDB e a utilização de uma modelagem híbrida complementam a arquitetura, permitindo explorar diferentes formas de representação e consulta dos dados. Dessa forma, as implementações e os testes descritos evidenciam o atendimento aos requisitos técnicos apresentados para a Sprint 4, com atenção à integridade, à rastreabilidade e à proteção das informações.